

LAMP: ASP Rule Induction and Optimization with LLMs

David Bunker
University of Potsdam



Headline

LAMP asks an LLM to recover ASP rules from *instance facts* and an *expected answer set*, then checks every candidate with CLINGO. Across 120 synthetic rule-induction benchmarks, the strongest LAMP models nearly matched ILASP, solved every ILASP timeout case, and usually produced smaller, more stratified programs than the synthetic originals.

99.2% Best final accuracy with LAMP <i>gpt-5-mini</i> : 119 / 120	98.3% Best local model result <i>gpt-oss:20b</i> : 118 / 120	3 / 3 ILASP timeout benchmarks solved by both strong LAMP models	3–6 Average output literals for correct solutions vs. 22 or 33 originals
--	---	---	---

Problem Setting

Writing ASP that is both correct and efficient is hard. LAMP studies a focused version of that challenge: recover a rule set from example facts and the stable model it should produce.

$$AS(F \cup \Pi) = \{A^*\}$$

Here F is the given fact set, A^* is the target answer set, Φ is a bounded hypothesis space, and the goal is to find a small program $\Pi \subseteq \Phi$ that reproduces A^* exactly. Candidate programs use safe unary normal rules with bounded positive and default-negated body literals. Safe means every variable appears in a positive body literal, so grounding is finite. No new facts are allowed.

- Both LAMP and ILASP receive the same facts, target answer set, and rule bounds.
- This makes the comparison about *search strategy*: LLM-guided generation versus conflict-driven symbolic search.
- The task is intrinsically difficult: bounded-arity non-ground normal ASP reasoning is Σ_2^P -complete, and learning adds another search layer.

Toy Benchmark

Facts	<code>d0(o0).</code> <code>d0(o1).</code> <code>d1(o1).</code>
Target answer set	<code>{ d0(o0), d0(o1), d1(o1), d2(o0) }</code>
One minimal solution	<code>d2(X) :- d0(X), not d1(X).</code>

Correctness is judged by the stable model, not by reproducing the exact synthetic source rules. Many different rule sets can yield the same answer set.

Benchmark Design

Parameter	Values used
Predicates	6, 10
Terms	6, 10
Facts	5, 6, 9
Rules	11
Overall literals	2, 3 per rule body
Negative literals	0, 1, 2
Minimum derived atoms	1, 3

- 120 valid synthetic combinations were generated.
- Programs use unary predicates and normal rules only, which keeps the comparison controlled while still requiring nontrivial induction.
- Each benchmark has exactly one answer set and at least one derived atom.

Why LAMP?

- ASP is powerful for configuration, planning, and combinatorial reasoning, but writing the rules is often the bottleneck.
- LAMP uses the LLM to generate ASP rules directly from facts, a target answer set, and rule bounds.
- This differs from using LLMs mainly as a front end for natural-language translation or ILASP task construction.
- Every candidate is checked by CLINGO at inference time, without LLM fine-tuning.
- Because the task is rule induction rather than text translation, success is evidence of structured reasoning rather than prompt memorization.

ILASP Baseline

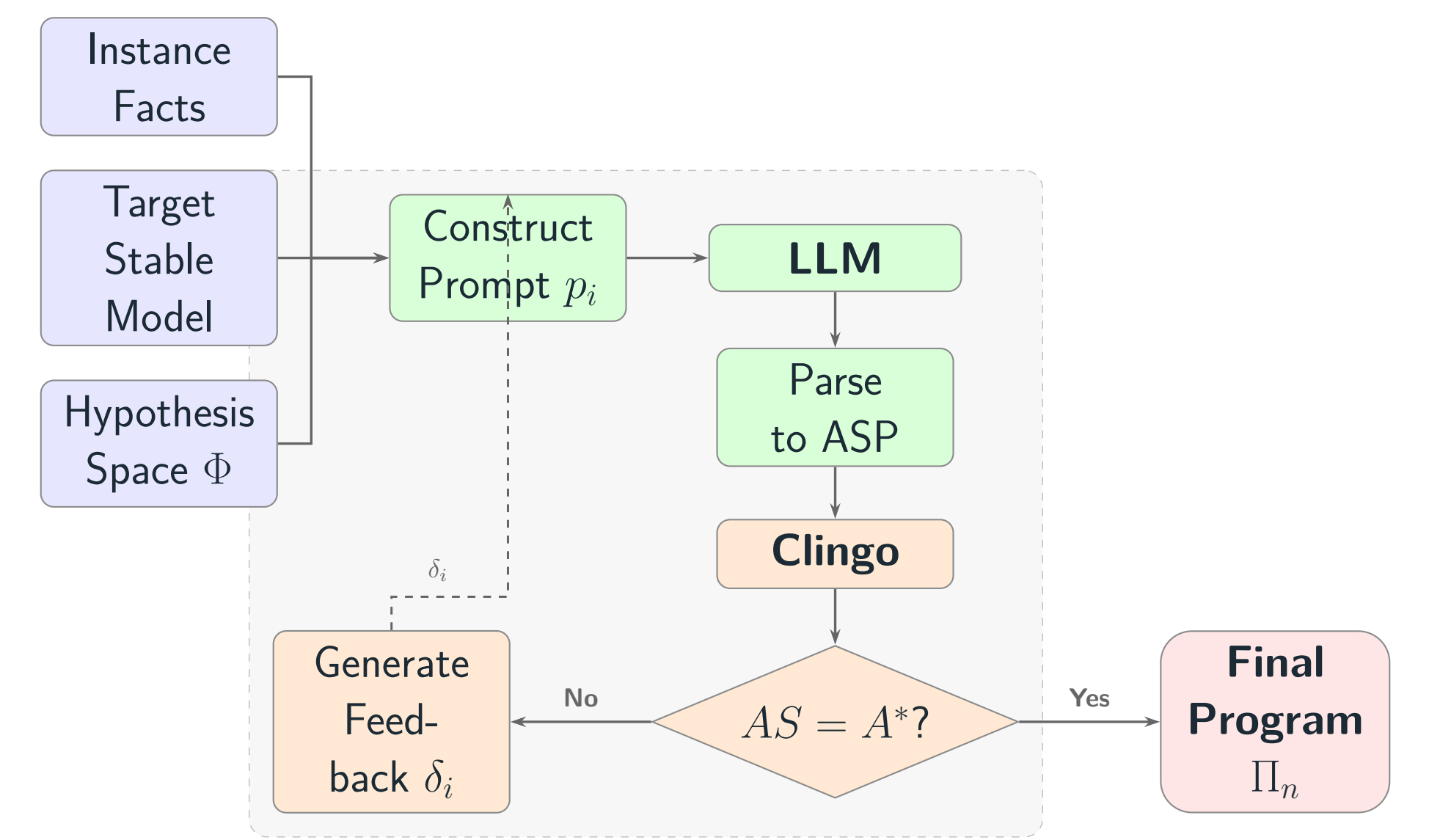
- ILASP solves the same task under the Learning from Answer Sets framework, but does so with conflict-driven symbolic search over an ASP metaprogram rather than iterative LLM generation.
- The hypothesis space is fixed by a mode bias derived from the benchmark profile.
 - The target answer set is encoded as a single context-dependent positive example.
 - In this study ILASP is the main symbolic baseline, run with a 420s timeout.

Why Hybridize?

- ILASP gives the strong symbolic baseline: usually sub-second, concise, and principled within the mode bias.
- LAMP is slower, but its token-level generation explores the space differently and solved all 3 ILASP timeout cases.
- The natural next system is not LLM versus solver: it is ILASP-style search with solver-checked LLM proposals when search stalls.

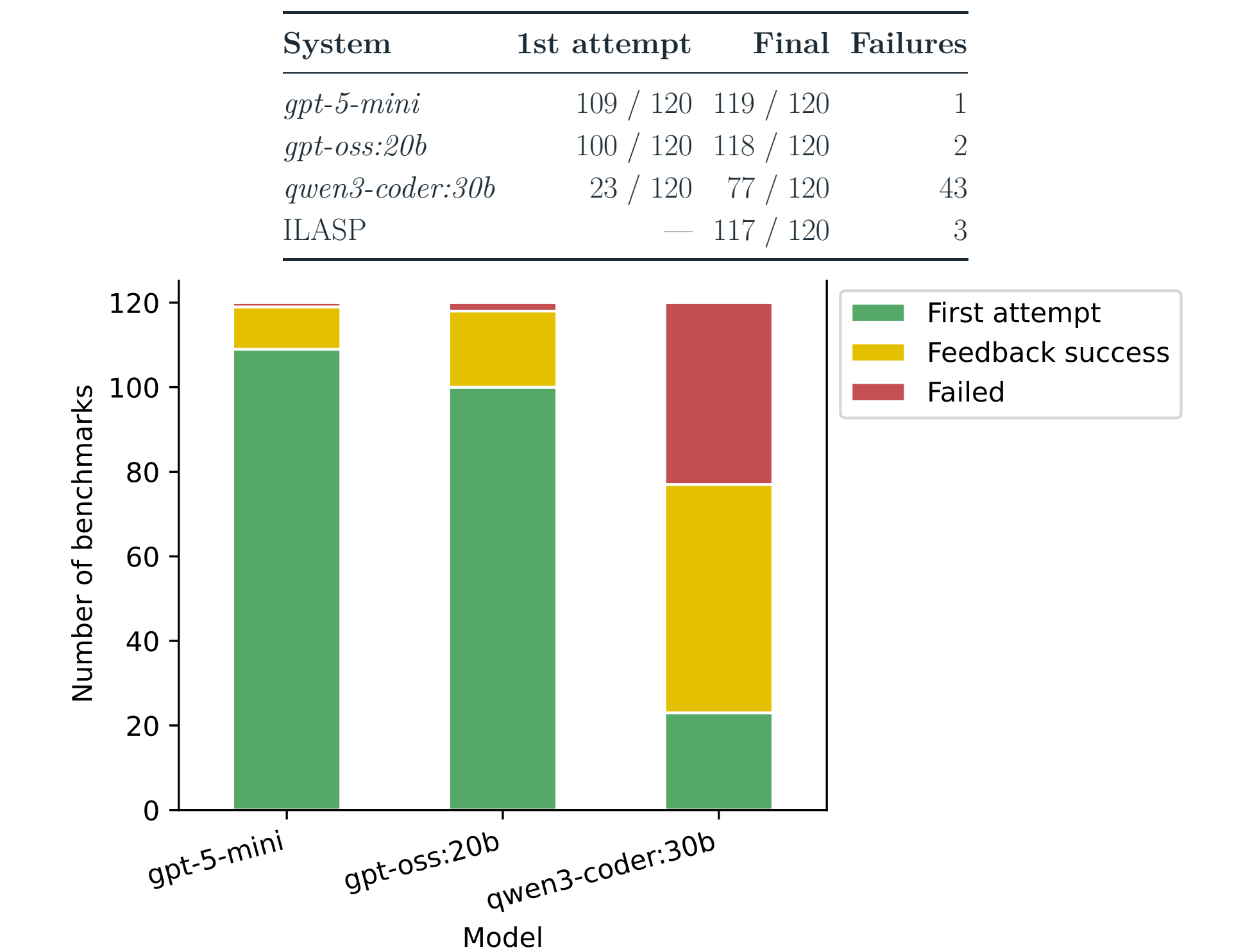
LAMP Protocol

- Prompt the LLM with instance facts F , target answer set A^* , rule schema bounds, and a worked example.
- Parse the raw output into an ASP candidate program Π_i .
- Run $F \cup \Pi_i$ with CLINGO.
- If the produced answer set mismatches A^* , feed the mismatch back and ask for a corrected rule set.
- Stop after success or after 3 total attempts.



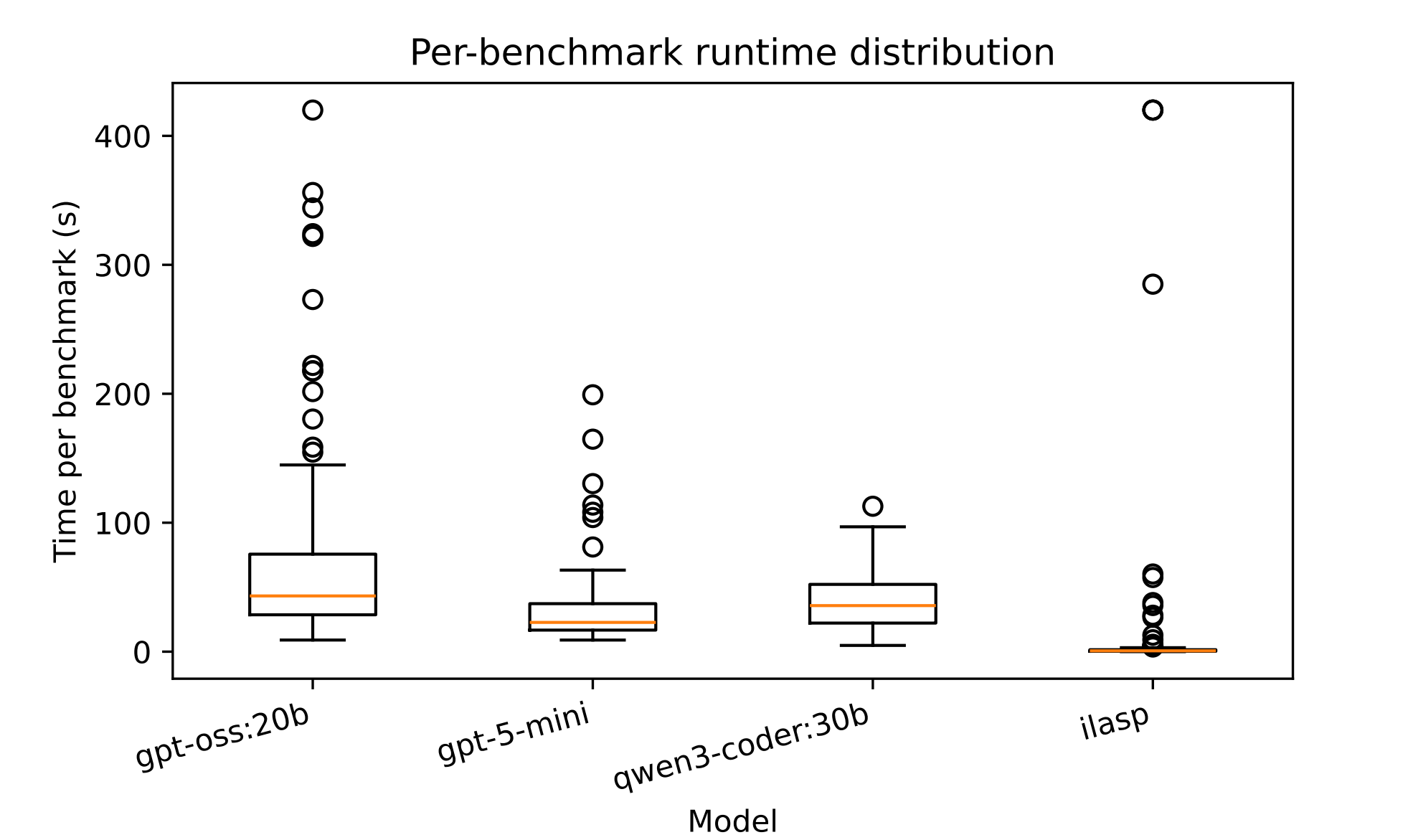
The key design choice is simple: let the LLM propose rules, but let a solver decide whether those rules are actually correct.

Accuracy Gains From Feedback



Feedback recovered 10 extra cases for *gpt-5-mini*, 18 for *gpt-oss:20b*, and 54 for *qwen3-coder:30b*. The loop matters, especially once the first draft is close but not exact.

Runtime And Search Behavior



Observation	Takeaway
ILASP median runtime: 0.66s	Conflict-driven search is usually extremely fast.
ILASP failures: 3, all at 420s	Rare search-space cliffs still matter.
Both strong LAMP models solved all 3 timeout cases	LLM-guided generation explores the space differently.

LLM median runtimes were 22.7s for *gpt-5-mini*, 43.2s for *gpt-oss:20b*, and 35.7s for *qwen3-coder:30b*. The *gpt-5-mini* timing excludes network latency.

Program Quality

Mean output literal count $L(\Pi)$ for correct solutions				
Original	<i>gpt-5</i>	<i>gpt-oss</i>	<i>qwen</i>	ILASP
22 literals	4.81	5.15	5.85	3.67
33 literals	3.49	3.82	3.84	2.71

Stratified solved programs	
System	Stratified / Solved
Original benchmarks	49 / 120
<i>gpt-5-mini</i>	119 / 119
<i>gpt-oss:20b</i>	117 / 118
<i>qwen3-coder:30b</i>	77 / 77
ILASP	117 / 117

Both ILASP and strong LAMP models compressed the synthetic source programs heavily. Correct outputs usually needed only a handful of literals, and almost every solved program became stratified even when the original benchmark was not.

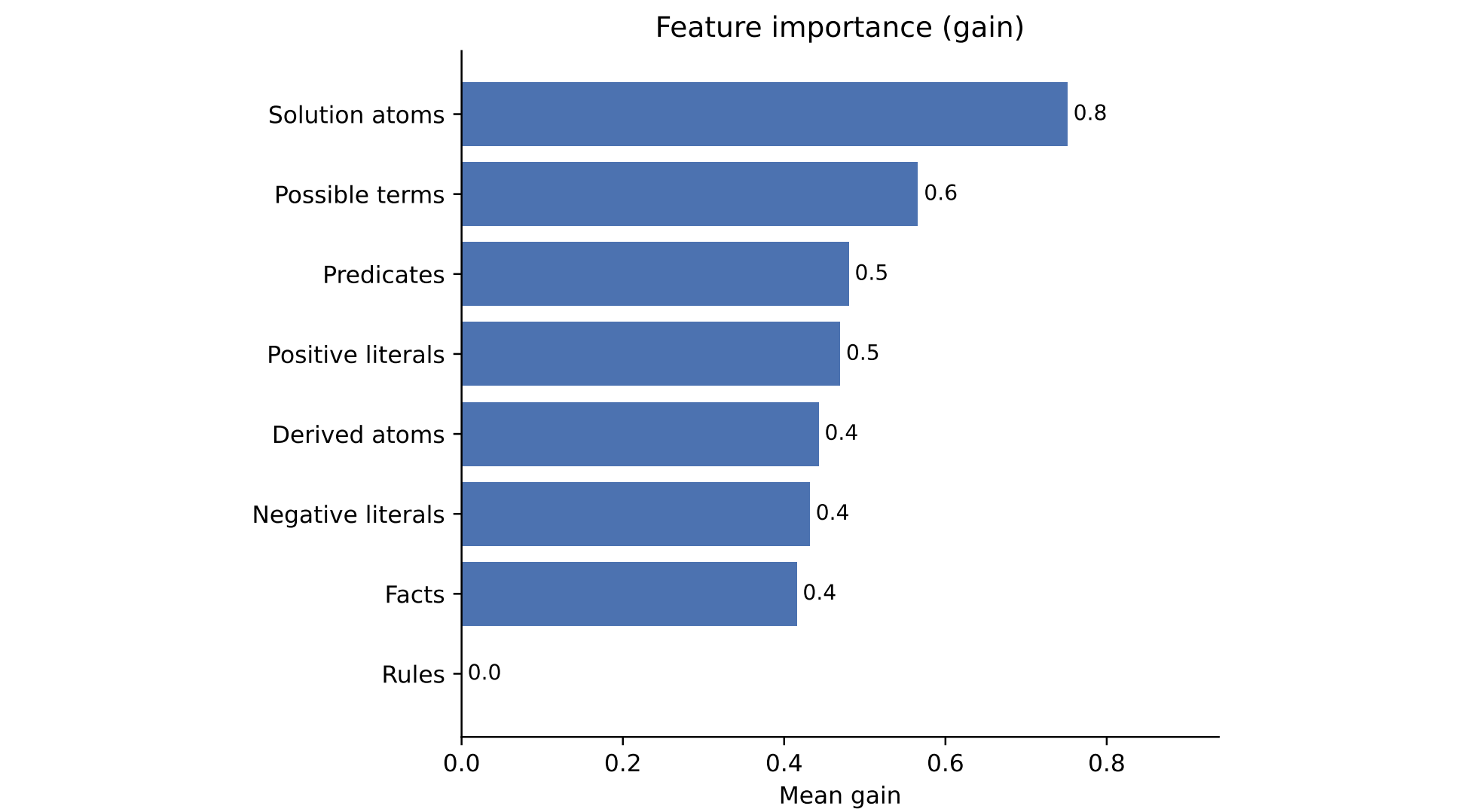
Timeout Cases

Benchmark	<i>gpt-5</i>	<i>gpt-oss</i>	<i>qwen</i>	ILASP
7	✓	✓	×	420s
27	✓	✓	×	420s
67	✓	✓	×	420s

The three ILASP failures were all timeouts, not logical contradictions. Each timeout case had 11 negative literals, and grounded-rule counts between 20 and 27 compared with a dataset average of 13.28. That points to search-space blowup rather than impossibility of the task.

Feature Analysis (RQ3)

The paper also asks which benchmark features are associated with LLM failure. The strongest models missed too few cases for a useful classifier, so the analysis focuses on *qwen3-coder:30b*.



The top gain features describe scale and vocabulary breadth: larger target answer sets, more possible terms, and more predicates make exact rule recovery harder. XGBoost reached 61.7% cross-validation accuracy with AUC 0.619, below the 64.2% majority baseline, so these are descriptive correlates rather than reliable predictors.

Take-home / Answers

- RQ1:** Solver-in-the-loop prompting can produce correct ASP rules from facts and a target answer set.
- RQ2:** LAMP nearly matches ILASP in accuracy; ILASP is faster on average, but LAMP solves the ILASP timeout cases.
- RQ3:** Failure correlates with target size and vocabulary breadth, but the current sample is too small for reliable prediction.
- Take-home:** LAMP is not a replacement for ILASP; it is a complementary LLM-guided search strategy that suggests hybrid symbolic-LLM systems.

Limits And Next Steps

- The study uses synthetic benchmarks with unary predicates, normal rules, and exactly one answer set.
- Each model was run once per benchmark at temperature 1, so small score gaps should be interpreted cautiously.
- Repeated trials, richer ASP constructs, ASPBench, planning benchmarks, and prompt optimization would strengthen the evaluation.
- The most interesting extension is a hybrid system: ILASP-style search plus an agentic LLM refinement loop.

Experimental Controls

Same input instance for both systems: facts, target answer set, and bounded rule schema. LAMP used 3 attempts, exact CLINGO stable-model matching, and the same 420s timeout as ILASP v4. *gpt-5-mini* used the OpenAI API; *gpt-oss:20b* and *qwen3-coder:30b* ran locally with Ollama. Source code, benchmark data, and generated ILASP tasks: <https://github.com/dbunker/lamp>.